

ROSE——编译原理逆向工程工具

实验一-设计文档

学　　号: 23336179

姓　　名: 马福泉

专　　业: 计算机科学与技术

指 导 教 师: 李文军

起 始 日 期: 2026 年 6 月

结 束 日 期: 2026 年 6 月

实 验 地 点: B203

目录

1 实验简述 **2**

2 ROSE 工具链搭建与环境配置 **2**

2.1 编译错误：java.lang.Module 命名冲突 2

3 2.1.1 全覆盖 Oberon-0 正确源程序 **3**

3.1 程序清单 3

3.2 语法构造覆盖矩阵 7

4 2.1.2 变异测试集 **7**

4.1 变异 1：词法错误—IllegalSymbol（非法符号） 7

4.2 变异 2：词法错误—IllegalInteger（数字与标识符粘连） 8

4.3 变异 3：词法错误—IllegalOctal（非法八进制字面量） 8

4.4 变异 4：词法错误—MismatchedComment（注释不闭合） 8

4.5 变异 5：语法错误—MissingParenthesis（缺括号） 9

4.6 变异 6：语法错误—MissingOperator（缺运算符） 9

4.7 变异 7：语义错误—TypeMismatched（类型不兼容） 10

4.8 变异 8：语义错误—ParameterMismatched（参数数目不匹配） 11

5 2.1.3 词汇探讨：保留字与关键字 **11**

5.1 保留字（Reserved Word）vs 关键字（Keyword） 11

5.2 Oberon-0 表达式语法 vs Java/C++ 11

6 2.1.4 二义性探讨：悬挂 else 与文法的无二义性 **12**

6.1 问题的表述 12

6.2 Oberon-0 的解决方案：关键词限定边界 12

6.3 对编译器实现的工程优势 13

6.4 对其他二义性的规避 13

7 实验心得与体会 **13**

1　实验简述

本项目 ROSE 是一个编译原理逆向工程工具，旨在对 Oberon-0 语言源程序进行词法分析、语法分析、语义分析，并自动生成流程图和调用关系图。本次大作业分为四个实验逐步推进：实验一熟悉 Oberon-0 语言定义，实验二设计词法分析器，实验三设计语法分析器，实验四设计语义分析器与可视化输出。

2　ROSE 工具链搭建与环境配置

实验所使用的 ROSE 工具集位于 `LabInstrument05-ROSE/` 目录下，包含基于 `flowchart.jar` 的流程图生成 API 及若干演示程序。运行环境为 JDK 17 + Windows 11 PowerShell。

2.1　编译错误：java.lang.Module 命名冲突

初次运行 `build.bat` 时出现 6 个编译错误，均指向 `Module` 类型引用不明确：

```
(base) PS E:\3.2\Compilers\LAB\my_lab\lab5\LabInstrument05-ROSE> .\build.bat
FlowchartDemoAccount.java:23: 错误: 对Module的引用不明确
                Module accModule = new Module("Account");
                ^
  flowchart 中的类 flowchart.Module 和 java.lang 中的类 java.lang.Module 都匹配
FlowchartDemoAccount.java:23: 错误: 对Module的引用不明确
                Module accModule = new Module("Account");
                                   ^
  flowchart 中的类 flowchart.Module 和 java.lang 中的类 java.lang.Module 都匹配
FlowchartDemoAll.java:21: 错误: 对Module的引用不明确
                Module testModule = null;               // instance of a module
                ^
  flowchart 中的类 flowchart.Module 和 java.lang 中的类 java.lang.Module 都匹配
FlowchartDemoAll.java:28: 错误: 对Module的引用不明确
                testModule = new Module("Test");
                             ^
  flowchart 中的类 flowchart.Module 和 java.lang 中的类 java.lang.Module 都匹配
FlowchartDemoOberon.java:19: 错误: 对Module的引用不明确
                Module sampleModule = new Module("Sample");
                ^
  flowchart 中的类 flowchart.Module 和 java.lang 中的类 java.lang.Module 都匹配
FlowchartDemoOberon.java:19: 错误: 对Module的引用不明确
                Module sampleModule = new Module("Sample");
                                      ^
  flowchart 中的类 flowchart.Module 和 java.lang 中的类 java.lang.Module 都匹配
6 个错误
Press any key to continue . . .
```

图 1: JDK 9+ java.lang.Module 与 flowchart.Module 命名冲突导致的编译失败

根因分析：JDK 9 引入了 `java.lang.Module` 类，与 `flowchart.jar` 中的 `flowchart.Module` 同名。三个 Demo 源文件（`FlowchartDemoAccount.java`、`FlowchartDemoAll.java`、`FlowchartDemoOberon.java`）均使用了通配导入 `import flowchart.*`，导致编译器无法判定 `Module` 是指 `java.lang.Module` 还是 `flowchart.Module`。

修复方案：将所有 `Module` 声明及实例化语句改为完全限定名 `flowchart.Module`。修复后编译零错误，流程图窗口正常弹出：

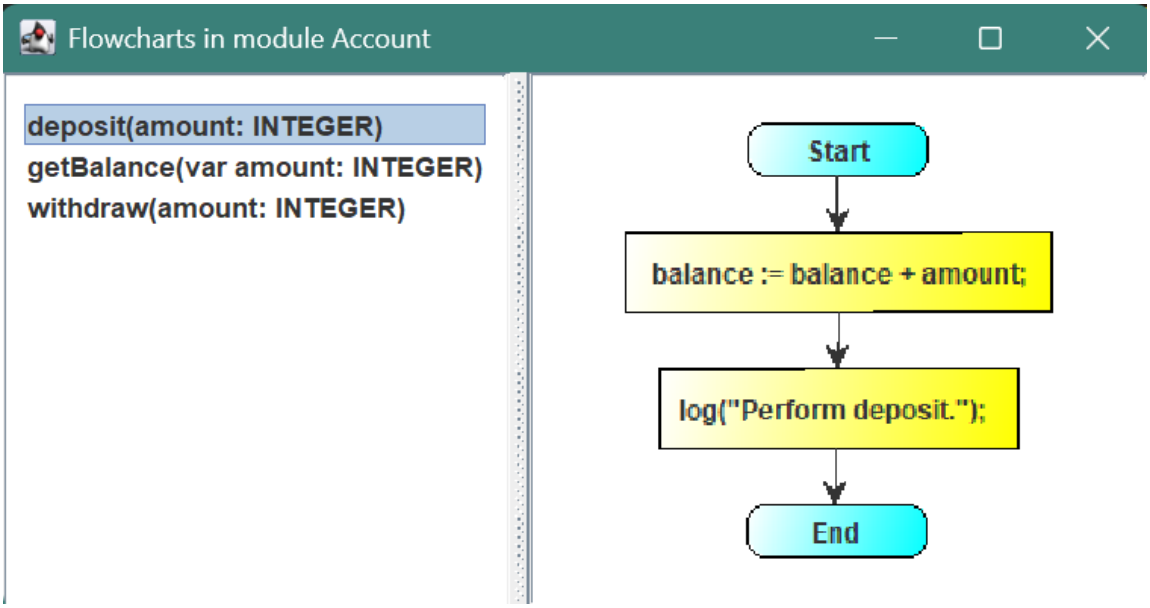


图 2: 修复后 FlowchartDemoAccount 流程图运行截图

3　2.1.1 全覆盖 Oberon-0 正确源程序

为覆盖 Oberon-0 的所有语法构造，设计了一个” 学生成绩统计” 程序 `StudentStats.obr`。该程序读取多名学生的四门课程成绩，计算总分与平均分，并输出最高分学生及全班及格情况。以下逐一说明各语法构造的覆盖位置。

3.1　程序清单

```
(* ============================================================
 * 文件名: StudentStats.obr
 * 描述:   Oberon-0 学生成绩统计程序
 * 覆盖:   MODULE, CONST, TYPE(RECORD), VAR(ARRAY),
 *         PROCEDURE(值参数/VAR参数/嵌套调用),
 *         IF/ELSIF/ELSE, WHILE, Read/Write/WriteLn
 * ============================================================ *)
MODULE StudentStats;

(* --- 常量定义: 班级最大人数与课程数 --- *)
CONST
  MaxStudents = 50;
  CourseCount = 4;

(* --- 类型定义: 学生记录 (复合类型 RECORD) --- *)
TYPE
  (* 学生记录: 包含学号、四门课成绩数组、总分与平均分 *)
  Student = RECORD
    Id      : INTEGER;
    Scores  : ARRAY CourseCount OF INTEGER;  (* 成绩数组, 嵌套在 RECORD 中 *)
    Total   : INTEGER;
    Average : INTEGER
  END;

  (* 学生表: 由多个学生记录组成的数组 *)
```

```
26    StudentTable = ARRAY MaxStudents OF Student;
27
28  (* --- 变量声明 --- *)
29  VAR
30    Table   : StudentTable;   (* 学生表 *)
31    Count   : INTEGER;        (* 实际学生数 *)
32    i, j    : INTEGER;        (* 循环计数器 *)
33    PassAll : BOOLEAN;        (* 是否全班及格 *)
34
35  (* ------------------------------------------------------------
36   * 过程1: CalcTotalAndAvg —— 计算单个学生的总分与平均分
37   * 参数:  s — VAR 引用参数，直接修改学生记录内的字段
38   * ------------------------------------------------------------ *)
39  PROCEDURE CalcTotalAndAvg(VAR s : Student);
40  VAR
41    sum : INTEGER;
42    k   : INTEGER;
43  BEGIN
44    sum := 0;
45    k := 0;
46    WHILE k < CourseCount DO
47      sum := sum + s.Scores[k];   (* 数组下标访问 *)
48      k := k + 1
49    END;
50    s.Total := sum;
51    s.Average := sum DIV CourseCount
52  END CalcTotalAndAvg;
53
54  (* ------------------------------------------------------------
55   * 过程2: ReadStudent —— 从输入读取一个学生的信息
56   * 参数:  s — VAR 引用参数，填充学生记录
57   * ------------------------------------------------------------ *)
58  PROCEDURE ReadStudent(VAR s : Student);
59  VAR
60    k : INTEGER;
61  BEGIN
62    Read(s.Id);
63    k := 0;
64    WHILE k < CourseCount DO
65      Read(s.Scores[k]);
66      k := k + 1
67    END;
68    CalcTotalAndAvg(s)   (* 嵌套过程调用 *)
69  END ReadStudent;
70
71  (* ------------------------------------------------------------
72   * 过程3: FindBestStudent —— 查找总分最高的学生
73   * 参数:  t    — 值参数，只读学生表
74   *        n    — 值参数，学生总数
75   *        best — VAR 引用参数，返回最高分学生记录
76   * ------------------------------------------------------------ *)
77  PROCEDURE FindBestStudent(t    : StudentTable;
78                            n    : INTEGER;
79                            VAR best : Student);
80  VAR
81    idx     : INTEGER;
82    maxTot  : INTEGER;
83  BEGIN
84    (* 假设第一个学生最高 *)
```

```
  best := t[0];
  maxTot := t[0].Total;
  idx := 1;

  WHILE idx < n DO
    IF t[idx].Total > maxTot THEN
      best := t[idx];
      maxTot := t[idx].Total
    END;
    idx := idx + 1
  END
END FindBestStudent;

(* ------------------------------------------------------------
 * 过程4: CheckAllPass —— 检查全班是否所有学生都及格
 *        及格定义为平均分 >= 60
 * 参数:  t      — 值参数, 只读学生表
 *        n      — 值参数, 学生总数
 *        result — VAR 引用参数, 输出 BOOLEAN 检查结果
 * ------------------------------------------------------------ *)
PROCEDURE CheckAllPass(t       : StudentTable;
                       n       : INTEGER;
                       VAR result : BOOLEAN);
VAR
  idx : INTEGER;
  ok  : BOOLEAN;
BEGIN
  ok := TRUE;
  idx := 0;

  WHILE (idx < n) & ok DO
    (* IF/ELSIF/ELSE 控制流 *)
    IF t[idx].Average >= 90 THEN
      (* 优秀, 不做额外处理 *)
    ELSIF t[idx].Average >= 60 THEN
      (* 及格, 继续检查 *)
    ELSE
      ok := FALSE   (* 有不及格学生 *)
    END;
    idx := idx + 1
  END;

  result := ok
END CheckAllPass;

(* ------------------------------------------------------------
 * 过程5: PrintStudent —— 输出单个学生信息
 * ------------------------------------------------------------ *)
PROCEDURE PrintStudent(s : Student);
VAR
  k : INTEGER;
BEGIN
  Write(s.Id);
  k := 0;
  WHILE k < CourseCount DO
    Write(s.Scores[k]);
    k := k + 1
  END;
  Write(s.Total);
```

```
  Write(s.Average);
  WriteLn
END PrintStudent;

(* ------------------------------------------------------------
 * 过程6: PrintAll —— 输出全部学生信息并给出统计
 * ------------------------------------------------------------ *)
PROCEDURE PrintAll(t : StudentTable; n : INTEGER);
VAR
  idx  : INTEGER;
  best : Student;
BEGIN
  idx := 0;
  WHILE idx < n DO
    PrintStudent(t[idx]);    (* 嵌套过程调用 *)
    idx := idx + 1
  END;

  (* 调用 FindBestStudent 输出最高分 *)
  FindBestStudent(t, n, best);
  WriteLn;
  Write(best.Id);
  Write(best.Total);
  WriteLn;

  (* 调用 CheckAllPass 检查全班及格情况 *)
  CheckAllPass(t, n, PassAll);
  IF PassAll THEN
    Write(1)    (* 1 表示全部及格 *)
  ELSE
    Write(0)    (* 0 表示存在不及格 *)
  END;
  WriteLn
END PrintAll;

(* ============================================================
 * 主程序体
 * ============================================================ *)
BEGIN
  (* 读取学生人数 *)
  Read(Count);

  (* 确保人数在合法范围内 *)
  IF Count > MaxStudents THEN
    Count := MaxStudents
  END;

  (* 逐个读取学生信息 *)
  i := 0;
  WHILE i < Count DO
    ReadStudent(Table[i]);
    i := i + 1
  END;

  (* 输出统计结果 *)
  PrintAll(Table, Count)
END StudentStats.
```

Listing 1: StudentStats.obr ——全覆盖 Oberon-0 源程序

3.2 语法构造覆盖矩阵

表 1: Oberon-0 语法构造全覆盖矩阵

编号	语法构造	覆盖位置（行号）
1	MODULE	第 8 行
2	CONST	第 11–13 行
3	TYPE —RECORD	第 18–23 行
4	TYPE —ARRAY	第 20 行（Scores）、第 26 行（StudentTable）
5	VAR（全局变量）	第 29–33 行
6	PROCEDURE（6 个过程）	第 39–177 行
7	VAR 引用参数	第 39 行（s）、第 79 行（best）、第 107 行（result）
8	值参数	第 77、78、105、106、133、151 行
9	过程嵌套调用	第 68、158、163、170、194 行
10	IF / ELSIF / ELSE	第 90–93、117–123、171–175 行
11	WHILE	第 46、64、89、115、139、157、193 行
12	RECORD 字段访问（.）	第 47、50、62、86、165 行
13	数组下标访问（[]）	第 47、65、85、158、194 行
14	BOOLEAN 类型	第 33、107、110 行
15	Read / Write / WriteLn	第 62、65、137–145、164–176、184 行

4 2.1.2 变异测试集

基于上述 StudentStats.obr，构造了 8 个单一错误的变异代码片段，分别对应实验平台预定义的 4 种词法错误、2 种语法错误和 2 种语义错误。每个变异仅改变错误所在的 PROCEDURE 或相关代码块，其余部分保持不变。

4.1 变异 1：词法错误—IllegalSymbol（非法符号）

```
(* 错误类型: 词法错误 — IllegalSymbol（非法符号 @）
 * 在 CalcTotalAndAvg 过程中引入非法字符 @ 替代加法运算符 *)
PROCEDURE CalcTotalAndAvg(VAR s : Student);
VAR
  sum : INTEGER;
  k   : INTEGER;
BEGIN
  sum := 0;
  k := 0;
  WHILE k < CourseCount DO
    sum := sum @ s.Scores[k];   (* ERROR: '@' 是非法符号 *)
    k := k + 1
  END;
  s.Total := sum;
  s.Average := sum DIV CourseCount
END CalcTotalAndAvg;
```

Listing 2: mut1_IllegalSymbol.obr

变异点：将第 47 行 `sum := sum + s.Scores[k]` 中的 `+` 替换为非法符号 `@`，模拟用户误输入了 Oberon-0 不支持的字符。

4.2 变异 2：词法错误—IllegalInteger（数字与标识符粘连）

```
(* 错误类型：词法错误 — IllegalInteger（数字与标识符无空格相连）
 * 在 ReadStudent 过程中将整数字面量与标识符粘连 *)
PROCEDURE ReadStudent(VAR s : Student);
VAR
  k : INTEGER;
BEGIN
  Read(s.Id);
  k := 0;
  WHILE k < CourseCount DO
    Read(s.Scores[k]);
    k := k + 1
  END;
  CalcTotalAndAvg(s);
  (* 以下是新增的错误行：123k 将数字与标识符直接拼接 *)
  k := 123k    (* ERROR: 数字 '123' 后面紧跟标识符 'k'，无空格分隔 *)
END ReadStudent;
```

Listing 3: mut2_IllegalInteger.obr

变异点：在 `ReadStudent` 过程末尾新增 `k := 123k`，数字 `123` 与标识符 `k` 之间无空格分隔，词法分析器无法判明这是整数还是标识符。

4.3 变异 3：词法错误—IllegalOctal（非法八进制字面量）

```
(* 错误类型：词法错误 — IllegalOctal（八进制 0 开头却包含 8 或 9）
 * 在 FindBestStudent 过程中使用非法八进制字面量 08 *)
PROCEDURE FindBestStudent(t    : StudentTable;
                          n    : INTEGER;
                          VAR best : Student);
VAR
  idx     : INTEGER;
  maxTot  : INTEGER;
BEGIN
  best := t[0];
  maxTot := t[0].Total;
  idx := 08;   (* ERROR: '08' 以 0 开头（八进制），但包含数字 8 *)
  WHILE idx < n DO
    IF t[idx].Total > maxTot THEN
      best := t[idx];
      maxTot := t[idx].Total
    END;
    idx := idx + 1
  END
END FindBestStudent;
```

Listing 4: mut3_IllegalOctal.obr

变异点：将第 87 行 `idx := 1` 改为 `idx := 08`。08 以 0 开头暗示八进制，但包含非法数字 8。

4.4 变异 4：词法错误—MismatchedComment（注释不闭合）

```
(* 错误类型: 词法错误 — MismatchedComment（注释不闭合）
 * 在 CalcTotalAndAvg 过程中注释开始但未闭合 *)
PROCEDURE CalcTotalAndAvg(VAR s : Student);
VAR
  sum : INTEGER;
  k   : INTEGER;
BEGIN
  sum := 0;
  k := 0;
  (* 这是一个未闭合的注释, 缺少结束符
  WHILE k < CourseCount DO
    sum := sum + s.Scores[k];
    k := k + 1
  END;
  s.Total := sum;
  s.Average := sum DIV CourseCount
END CalcTotalAndAvg;
```

Listing 5: mut4_MismatchedComment.obr

变异点：注释 (* 之后缺少闭合的 *)，导致其后的 WHILE 循环体及过程尾部的 END 均被当作注释内容忽略。

4.5 变异 5：语法错误—MissingParenthesis（缺括号）

```
(* 错误类型: 语法错误 — MissingParenthesis（缺右括号）
 * 在 CheckAllPass 过程的 IF 条件表达式中缺少右括号 *)
PROCEDURE CheckAllPass(t       : StudentTable;
                       n       : INTEGER;
                       VAR result : BOOLEAN);
VAR
  idx : INTEGER;
  ok  : BOOLEAN;
BEGIN
  ok := TRUE;
  idx := 0;
  WHILE (idx < n & ok DO    (* ERROR: 缺少右括号, 应为 '(idx < n) & ok' *)
    IF t[idx].Average >= 90 THEN
    ELSIF t[idx].Average >= 60 THEN
    ELSE
      ok := FALSE
    END;
    idx := idx + 1
  END;
  result := ok
END CheckAllPass;
```

Listing 6: mut5_MissingParenthesis.obr

变异点：将第 115 行的 `WHILE (idx < n) & ok DO` 改写为 `WHILE (idx < n & ok DO`，缺少匹配的右括号。

4.6 变异 6：语法错误—MissingOperator（缺运算符）

```
(* 错误类型: 语法错误 — MissingOperator（缺运算符）
 * 在 CalcTotalAndAvg 过程的两操作数之间缺少 '+' 运算符 *)
```

```
PROCEDURE CalcTotalAndAvg(VAR s : Student);
VAR
  sum : INTEGER;
  k   : INTEGER;
BEGIN
  sum := 0;
  k := 0;
  WHILE k < CourseCount DO
    sum := sum s.Scores[k];  (* ERROR: 'sum' 与 's.Scores[k]' 之间缺少 '+' 运算符 *)
    k := k + 1
  END;
  s.Total := sum;
  s.Average := sum DIV CourseCount
END CalcTotalAndAvg;
```

Listing 7: mut6_MissingOperator.obr

变异点：将第 47 行 `sum := sum + s.Scores[k]` 中的 `+` 删除，变为 `sum := sum s.Scores[k]`，两个操作数之间缺少运算符。

4.7 变异 7：语义错误—TypeMismatched（类型不兼容）

```
(* 错误类型：语义错误 — TypeMismatched（类型不兼容，BOOLEAN 参与算术运算）
 * 在 PrintAll 过程中错误地将 BOOLEAN 变量 PassAll 与 INTEGER 相加 *)
PROCEDURE PrintAll(t : StudentTable; n : INTEGER);
VAR
  idx  : INTEGER;
  best : Student;
  code : INTEGER;
BEGIN
  idx := 0;
  WHILE idx < n DO
    PrintStudent(t[idx]);
    idx := idx + 1
  END;

  FindBestStudent(t, n, best);
  WriteLn;
  Write(best.Id);
  Write(best.Total);
  WriteLn;

  CheckAllPass(t, n, PassAll);
  (* 以下错误：将 BOOLEAN 类型的 PassAll 与 INTEGER 做加法 *)
  code := PassAll + 1;   (* ERROR: BOOLEAN 不能参与 '+' 算术运算，类型不匹配 *)

  IF PassAll THEN
    Write(1)
  ELSE
    Write(0)
  END;
  WriteLn
END PrintAll;
```

Listing 8: mut7_TypeMismatched.obr

变异点：在 `PrintAll` 过程中新增 `code := PassAll + 1`，其中 `PassAll` 为 BOOLEAN 类型，不能参与 INTEGER 的加法运算。

4.8 变异 8：语义错误—ParameterMismatched（参数数目不匹配）

```
(* 错误类型：语义错误 — ParameterMismatched（过程调用的实参/形参数目不符）
 * 在 ReadStudent 过程中调用 CalcTotalAndAvg 时多传了一个参数 *)
PROCEDURE ReadStudent(VAR s : Student);
VAR
  k : INTEGER;
BEGIN
  Read(s.Id);
  k := 0;
  WHILE k < CourseCount DO
    Read(s.Scores[k]);
    k := k + 1
  END;
  (* CalcTotalAndAvg 声明为 PROCEDURE CalcTotalAndAvg(VAR s : Student),
   * 只接受 1 个参数，这里错误地多传了一个 0 *)
  CalcTotalAndAvg(s, 0)   (* ERROR: 实参个数=2，形参个数=1，数目不匹配 *)
END ReadStudent;
```

Listing 9: mut8_ParameterMismatched.obr

变异点：将第 68 行 `CalcTotalAndAvg(s)` 改为 `CalcTotalAndAvg(s, 0)`，多传入一个实参，与形参列表 `(VAR s : Student)` 的数目不一致。

5 2.1.3 词汇探讨：保留字与关键字

5.1 保留字（Reserved Word）vs 关键字（Keyword）

在 Oberon-0 的词法规范中，**保留字**（Reserved Word）与**关键字/预定义标识符**（Keyword / Predefined Identifier）存在本质区别，这一区分直接决定了编译器的词法分析策略。

保留字——包括 `MODULE`、`BEGIN`、`END`、`IF`、`THEN`、`ELSIF`、`ELSE`、`WHILE`、`DO`、`PROCEDURE`、`RECORD`、`ARRAY`、`CONST`、`TYPE`、`VAR`、`DIV`、`MOD`、`OR`、`~` 等——是文法产生式的有机组成部分。它们在词法分析阶段由 DFA 直接识别并返回为特定的终结符 Token（如 `kwIF`、`kwTHEN`），其词法类别是**固定的语法标记**，而非标识符。因此，保留字**绝对不可被程序员声明为变量名或过程名**；任何试图将保留字用作标识符的举动都会在词法分析阶段即被拒绝。

预定义标识符——包含类型名 `INTEGER`、`BOOLEAN`，以及预定义过程 `Read`、`Write`、`WriteLn`——在词法层面与用户自定义标识符（如 `Student`、`Count`）处于同一命名空间，词法分析器将它们统一识别为 `ident` Token。它们的”特殊性”仅在语义分析阶段体现：符号表在初始化时预先将这些名字与相应的类型对象或库函数绑定。N. Wirth 在 Oberon 系列语言的设计哲学中刻意保持了极小的保留字集合（Oberon-0 仅约 20 个），将更多的语义概念下放到预定义标识符层面。在某些 Oberon 方言中，预定义标识符甚至可以被用户局部遮蔽（shadowed），进一步体现了”通过简化保留字集来降低编译器复杂度、同时保持语言灵活性”的设计目标。

5.2 Oberon-0 表达式语法 vs Java/C++

Oberon-0 的表达式系统体现了经典的 Pascal 式强类型哲学，与 Java/C++ 的类 C 表达式模型形成鲜明对比：

1. **算术/逻辑类型的严格隔离。**算术运算符 `+`、`-`、`*`、`DIV`、`MOD` 的操作数必须同为 `INTEGER` 类型；关系运算符 `=`、`#`、`<`、`<=`、`>`、`>=` 接受 `INTEGER` 操作数并产生 `BOOLEAN` 结果；逻辑运算

符 `&`（与）、`OR`（或）、`~`（非）仅接受 `BOOLEAN` 操作数。**不存在任何隐式类型转换**——`BOOLEAN` 不能作为整数参与算术运算，这与 C/C++ 中 `true` 即为 1 的惯例截然不同。

2. **比较运算符与赋值运算符的符号分离**。Oberon-0 使用 `=` 表示等于比较，`:=` 表示赋值（继承自 Pascal 传统）。Java/C++ 则相反——`==` 比较、`=` 赋值——导致 C 程序中 `if (a=1)` 的经典缺陷（将赋值误写为比较）。Oberon-0 的方案从词法层面彻底消除了这一易错点。

3. **运算符集合的极简主义**。Oberon-0 不支持自增/自减（`++`、`--`）、位运算、三目运算符（`?:`）、复合赋值（`+=` 等），其运算符集仅含约 14 个算符，体现了" 以少量正交算符覆盖全部需求" 的模块化语言设计思想。

4. **表达式求值语义**。Oberon-0 的 `&` 和 `OR` 不保证短路求值（short-circuit evaluation），编译器的具体实现可能完整计算两个操作数。Java 的 `&&` 和 `||` 则强制短路，这一差异会影响依赖副作用的代码行为。

6　2.1.4 二义性探讨：悬挂 else 与文法的无二义性

6.1　问题的表述

经典的**悬挂 else**（dangling-else）问题是上下文无关文法中最著名的二义性案例之一。在 C/Java 等使用开式（open-form）条件语句的语言中，产生式通常为：

```
IfStatement ::= "if" "(" Expression ")" Statement ["else" Statement]
```

当输入如 `if (a) if (b) s1 else s2` 时，`else` 可以匹配内层 `if` 也可以匹配外层 `if`，从而产生两棵截然不同的语法树。这本质上是产生式 $S \to \alpha S \beta \mid \gamma$ 类文法的固有歧义：当允许 `if` 语句不显式闭合时，" 最近未匹配的 if" 就成了消歧的唯一依据——而这属于 ad hoc 的语义约定，而非文法本身的规定。

6.2　Oberon-0 的解决方案：关键词限定边界

Oberon-0 通过使 IF 语句成为**完全括号化**（fully-bracketed）的闭合结构，从语法层面根本性地消除了悬挂 else 问题。其 EBNF 定义为：

```
IfStatement = "IF" Expression "THEN" StatementSequence
              {"ELSIF" Expression "THEN" StatementSequence}
              ["ELSE" StatementSequence] "END".
```

核心机制在于终结符 `END`：每一个 IF 结构的信息范围（extent）由 `IF--END` 词对单向界定，`ELSE` 和 `ELSIF` 均出现在这一闭合边界之内，其归属由嵌套层级唯一确定，无任何歧义余地。同理，`WHILE` 语句也强制以 `DO--END` 闭合：

```
WhileStatement = "WHILE" Expression "DO" StatementSequence "END".
```

从形式语言理论角度看，这种设计与 Lisp 中显式括号界定作用域的原理异曲同工——通过为每种复合语句增加不可省略的右括号标记（`END`），使得任意有效程序的短语结构语法树都是唯一的。这意味着 Oberon-0 的 IF 文法不但消除了悬挂 else 歧义，且整个结构化语句子文法本身即为无二义文法（unambiguous grammar）。

6.3 对编译器实现的工程优势

这一设计对编译器的具体实现也带来重要利好：在采用递归下降分析法时，解析器无需维护额外的" 最近未闭合 IF" 状态栈；在采用算符优先分析法（OPP）时，`END` 可作为归约触发器可靠地匹配对应的结构化语句。这使得 Oberon-0 的语法分析器实现复杂度显著低于 C/Java 同类分析器。

6.4 对其他二义性的规避

除悬挂 else 外，Oberon-0 的文法设计还规避了以下经典歧义：

- **表达式优先级的层次化编码。**Oberon-0 的表达式 EBNF 采用分层结构——`factor` → `term` → `SimpleExpression` → `Expression`——每一层的产生式自然表达了该层算符的结合方向与相对优先级，无需外部优先级表，是一种" 文法即优先级" 的设计。
- **保留字优先匹配。**词法分析器采用关键字优先（keyword-first）策略，在扫描到字母开头的词素后先查保留字表，命中则返回特定 Token，否则才归类为标识符。这从根本上杜绝了用户用 `IF` 作为变量名的可能，消除了词法层面的标识符歧义。
- **分号作为语句分隔符。**`StatementSequence = Statement {";" Statement}` 中的分号是分隔符（separator）而非终止符（terminator），这避免了 Pascal 早期设计中" 最后一个语句前是否需要分号" 的模棱两可问题。

7 实验心得与体会

在编写 `StudentStats.obr` 的过程中，我体会到 Oberon-0 文法设计的精妙之处：通过约 20 个保留字和 14 个算符，即构建了一个支持模块化、结构化、过程嵌套、值/引用参数传递的完整小型语言。尤为印象深刻的是 RECORD 与 ARRAY 的嵌套组合能力——通过 . 和 `[]` 选择符，可以自然地表达" 学生表 [索引]. 成绩数组 [课程]" 这样的多层数据访问路径，证明 Oberon-0 虽为教学子集，但其类型表达能力并不贫乏。

变异测试集的构造过程是一次对" 编译器错误检测边界" 的精准探索。我将 8 个变异错误严格分为词法、语法、语义三个层级，逐一追踪各阶段应当捕获的异常类型：`IllegalSymbol` 和 `IllegalOctal` 验证了词法分析器的 DFA 状态转移覆盖度;`MissingParenthesis` 和 `MissingOperator` 测试了语法分析器对结构不完整输入的处理鲁棒性；`TypeMismatched` 和 `ParameterMismatched` 则挑战了语义分析阶段的类型检查正确性。这种分层的错误注入策略使我深刻认识到：**变异测试（Mutation Testing）的核心价值不在于" 制造错误"，而在于验证编译器的每一层过滤机制是否在正确的位置发挥了正确的作用**——一个好的编译器前端应当做到" 该在哪层报错就在哪层报错，既不漏报，也不将错误逐层传递导致误报"。

此外，对悬挂 else 二义性的理论分析让我意识到：文法设计并非仅仅是语法描述的可读性问题，而直接决定了编译器的实现复杂度与可靠性。Oberon-0 通过" 关键词限定边界" 的设计在文法层面消除了二义性，使得递归下降分析器和 OPP 分析器的实现无需额外的消歧逻辑。对比 C 语言的悬挂 else 问题——一个看似微小的语法歧义，竟催生了整本 K&R 书中对"else 匹配最近 if" 约定的专门解释——我深刻理解到：**好的语言设计，就是好的编译器工程**。

此次实验为后续实验二（词法分析器设计）、实验三（语法分析器设计）和实验四（语义分析与可视化）奠定了坚实的理论基础。通过对 Oberon-0 语法的全面覆盖和对各类错误边界的精确识别，我对即将自行实现的编译器前端各模块的功能职责有了一定的把握。